

基于区块链的智慧教育 学术诚信保障系统

——EduChain 系统设计与实现

摘要 学术诚信是高等教育面临的重大挑战。传统作业评审依赖教师人工批改和集中化查重工具,存在评审效率低、标准不统一、结果不可审计等问题。本文提出了一种基于区块链的智慧教育学术诚信保障系统(EduChain),将AI大语言模型评审、博弈论经济激励和区块链存证技术有机融合。系统采用“乐观执行”机制——默认信任AI评审结果、直接完成学分结算,仅在有异议时触发沙箱代码验证和学术仲裁委员会投票裁决。EduChain包含六个核心阶段:教师发布作业、学生提交承诺、AI+同行评审、争议窗口、仲裁裁决和学分结算。系统由四层架构组成:Solidity智能合约层、Python FastAPI后端服务层、React TypeScript前端交互层和Foundry沙箱执行层。经济模型通过StakeOracle动态计算学分质押门槛,确保诚实提交和公正评审是所有参与者的严格占优策略。实验结果表明,EduChain在抄袭检测精确率(0.89)和召回率(0.85)上分别优于单AI代理基线23.6%和18.1%,乐观路径相比全量验证方案降低了约67%的链上Gas消耗。消融实验进一步验证了争议挑战机制和乐观设计各自对系统性能的贡献。

关键词 智慧教育;学术诚信;区块链存证;AI作业评审;博弈论;乐观执行;抄袭检测

EduChain: Blockchain-based Smart Education Academic Integrity Assurance System

Abstract Academic integrity is a major challenge facing higher education. Traditional assignment review relies on manual grading and centralized plagiarism detection tools, suffering from low efficiency, inconsistent standards, and unauditable results. This paper proposes EduChain, a blockchain-based smart education academic integrity assurance system that organically integrates AI large language model review, game-theoretic economic incentives, and blockchain notarization. The system adopts an optimistic execution mechanism—trusting AI review results by default for direct credit settlement, triggering sandbox code verification and academic arbitration committee voting only when disputes arise. EduChain encompasses six core phases: assignment publication, student submission commitment, AI + peer review, dispute window, arbitration judgment, and credit settlement. The system consists of a four-layer architecture: Solidity smart contract layer, Python FastAPI backend service layer, React TypeScript frontend interaction layer, and Foundry sandbox execution layer. The economic model dynamically calculates credit staking thresholds through a StakeOracle, ensuring that honest submission and fair reviewing are strictly dominant strategies for all participants. Experimental results demonstrate that EduChain outperforms the single AI agent baseline by 23.6% in plagiarism detection precision (0.89) and 18.1% in recall (0.85), while the optimistic path reduces on-chain Gas consumption by approximately 67% compared to the full verification approach. Ablation studies further validate the respective contributions of the challenge mechanism and optimistic design to system performance.

Keywords smart education; academic integrity; blockchain notarization; AI assignment review; game theory; optimistic execution; plagiarism detection

1 引言

教育信息化和在线教学的快速发展使得编程类课程作业的评审成为一项巨大的挑战。全球范围内, Coursera、edX 等在线教育平台已累计超过 2 亿注册用户, 每年产生数千万份编程作业提交。与此同时, ChatGPT、GitHub Copilot 等 AI 代码生成工具的普及, 使得学生可以轻松生成高质量代码, 学术抄袭和作弊的形式愈发隐蔽和复杂。据统计, 2024 年美国高校报告的学术作弊案例中, 约 37% 涉及 AI 生成代码未被标注引用^[2]。

当前高校学术诚信保障主要依赖三种途径: (1) 教师人工批改, 虽能识别明显的抄袭痕迹, 但面对大规模班级时效率极低, 且评审标准因人而异; (2) 集中化查重系统 (如 Turnitin^[2]、MOSS^[2]), 能检测文本级别的相似度, 但对代码结构修改、变量重命名等混淆手段的检测能力有限; (3) 同行评议机制, 依赖学生互相评分, 但缺乏有效的激励约束, 容易流于形式或出现串通打分。

以上方案普遍存在三个核心问题: 评审结果的正确性依赖中心化权威且难以事后审计; 评审者的激励与诚实评审的目标不完全对齐——缺乏经济学层面的机制设计; 争议解决过程不透明——学生无法对评审结果提出有效的质疑和举证。这些问题催生了一个核心需求: **如何在去中心化环境下, 通过经济激励和密码学承诺, 使 AI 代理能够公正地完成编程作业评审, 并在出现学术争议时通过可信的仲裁机制达成共识。**

本文提出的 EduChain 智慧教育学术诚信保障系统针对上述问题提供了一个完整的解决方案。EduChain 的核心设计思想是“乐观执行”——系统默认信任 AI 评审和同行评分的结果, 直接完成学分分配和学术信誉更新; 只有当举报者发现疑似抄袭或错误并提出挑战时, 才触发代价较高的沙箱代码验证和学术仲裁委员会投票流程。这种设计使得绝大多数正常作业评审沿乐观路径快速完成 (约 70%), Gas 消耗大幅降低, 同时通过博弈论经济约束确保所有参与者没有作弊动机。

EduChain 的主要贡献包括: (1) 提出了首个面向编程教育场景的区块链学术诚信协议, 包含完整的乐观路径和争议路径双轨机制; (2) 设计了基于博弈论的教育经济模型, 通过动态学分质押计算确保诚实学习与公正评审的占优策略性; (3) 实现了 LLM 驱动的 AI 自动评审代理、沙箱隔离抄袭验证环境, 以及 commit-reveal 防串通同行评分机制; (4) 通过全面的对比实验验证了系统在抄袭检测精度、召回率和链上 Gas 效率方面的优势。

本文其余部分组织如下: 第 2 节分析教育学术诚信面临的核心问题与技术挑战; 第 3 节详细阐述

EduChain 的系统设计; 第 4 节介绍关键组件实现; 第 5 节呈现全面的实验评估结果, 包括主实验对比、消融实验、博弈论验证以及 11 项扩展实验 (抄袭类型细粒度分析、可扩展性、抗女巫攻击、跨 LLM 对比、挑战窗口敏感性、验证者机制分析、验证代码生成效率、参数敏感性、时间开销分解、学术声誉演化模拟和误报漏报成因分析); 第 6 节讨论相关工作; 第 7 节总结全文并展望未来方向。

2 核心问题与技术挑战

2.1 核心问题

EduChain 旨在解决的核心问题可以概括为: **如何在去中心化环境下, 构建一个经济激励相容、评审结果可验证、学术争议可仲裁的智慧教育学术诚信保障系统。**

具体而言, 该问题包含四个层面:

(1) **激励对齐问题。**在传统教育评审模式中, 评审者的努力程度与所得报酬之间缺乏严格的数学绑定。一个评审者可以选择“偷懒”——只做浅层检查, 节省时间和精力, 同时期望不会被发现。如果这种投机行为的期望收益超过诚实评审的收益, 理性参与者就会选择偷懒。因此, 需要设计一种经济机制, 使得诚实评审的期望收益严格大于偷懒评审的期望收益。

(2) **评审结果可验证性问题。**传统的作业评审结果通常以分数形式记录在教务系统中, 缺乏可验证的证据支撑。当学生对评分有异议或两个评审者给出不同结论时, 缺乏自动化的方式来验证谁是正确的。EduChain 通过语义状态承诺 (stateRoot/evidenceRoot/traceRoot) 和可执行的验证代码来解决这一问题——任何学术争议的指控必须附带可自动化验证的可执行证据。

(3) **学术争议的去中心化仲裁问题。**传统教育体系中, 学术争议由教务部门或教师个人裁决, 缺乏透明度和多方监督。需要设计一个去中心化的仲裁机制, 使得裁决结果可信、防操纵、可评审。同时, 仲裁过程必须高效——因为大多数情况下作业评审正常通过, 不应触发仲裁。

(4) **效率与安全的平衡问题。**如果每份作业都进行全量代码验证和多方评审, 系统安全性最高但链上 Gas 消耗巨大、处理速度极慢。如果完全不验证, 系统效率最高但学术诚信无法保证。需要在两者之间找到最优平衡点。

2.2 技术挑战

挑战一: 乐观执行中的争议证明机制。乐观执行的核心是“信任但可验证”。当举报者声称某份作业存在抄袭或错误时, 系统如何高效地验证举报者的声明? 这需要设计一种机制, 使得: (a) 举报者能够提

交可被自动执行的验证代码;(b)该验证代码可以在隔离沙箱中确定性重放;(c)重放结果可以作为链上仲裁的客观依据。

挑战二:博弈论经济约束的参数化设计。要确保诚实行为是占优策略,需要精确计算最低学分质押金额。这个计算依赖于多个概率参数——问题被检测的概率 p_{detect} 、仲裁正确的概率 p_{arb} 、诚实评审成本 C_a 和偷懒评审成本 C'_a 。这些参数在教育场景中需根据课程难度、作业类型等动态调整,以确保经济模型始终有效。

挑战三:AI 代理评审质量的客观评估。不同 AI 模型对同一份作业的评审结果可能存在显著差异。系统需要一种客观的评分机制来衡量评审质量,并在评分异常时自动触发惩罚。同时,评分机制本身需要防操纵——评审者可能串通给出不公正的分数以谋取学分奖励。

挑战四:沙箱执行的确定性隔离。验证代码可能包含恶意操作(如无限循环、资源耗尽)。沙箱环境需要确保:(a)代码执行不会影响主机系统;(b)执行结果是确定性的——任何人重放都应得到相同结果;(c)资源使用受到严格限制(时间、内存、网络访问)。

3 EduChain 系统设计

3.1 整体架构

EduChain 采用四层架构,如图 1所示。

[系统架构图:展示四层结构—前端交互层/后端服务层/区块链层/沙箱执行层]

图 1: EduChain 系统整体架构。自上而下分为四层:前端交互层(React TypeScript)、后端服务层(FastAPI + LLM Agent)、区块链层(Solidity 合约 + Anvil 本地链)、沙箱执行层(Foundry 隔离验证环境)。

前端交互层:基于 React 18 + TypeScript + Tailwind CSS 构建,提供监控面板(实时数据统计、

任务状态追踪、参与者列表、事件日志)和学术诚信操作台(AI 作业评审、抄袭验证证据生成、仲裁投票、同行评分、沙箱重放、一键评审流水线)。通过 ethers.js 与链上合约交互,通过 Vite 代理与后端 API 通信。

后端服务层:基于 FastAPI 构建的 Python 后端(端口 8000)。核心组件包括:(1) AutonomousAuditAgent——集成 LLM(DeepSeek)的自主评审代理,实现完整评审流水线:代码分析 → 问题识别 → 验证代码生成 → 语义根计算;(2) SandboxService——管理 Foundry 沙箱工作区,接收验证代码 + 学生作业,执行隔离验证并解析输出;(3) AgentService——代理生命周期管理单例。

区块链层:四个 Solidity 智能合约运行在 Anvil 本地以太坊兼容链(端口 8545)上。Registry 管理参与者注册和学术声誉;DisputeResolution 实现六阶段学术争议流水线;ArbitrationCommittee 处理 VRF 仲裁委员会选举和 EIP-712 多签投票;StakeOracle 执行博弈论学分质押参数动态计算。

沙箱执行层:基于 Foundry 测试框架的隔离验证环境。通过资源限制(2GB 内存、2 CPU、60 秒超时)、进程隔离和网络阻断确保验证代码的安全确定性执行——任何人重放同一验证代码都会得到相同结果。

3.2 六阶段学术争议流水线

EduChain 协议的核心是一个六阶段状态机,如图 2所示。每份作业评审任务按顺序经历以下阶段:

[六阶段流水线图:教师发布 → 学生提交 → AI+ 同行评审 → 争议窗口 → 仲裁裁决 → 学分结算]

图 2: EduChain 六阶段学术争议流水线。实线表示乐观路径(无争议,约占 70% 交易),虚线表示争议路径(有挑战)。

阶段 1: Publish (教师发布作业)。教师调用 DisputeResolution 合约的 publishTask() 函数,提交作业要求的代码哈希(codeHash)、评分标准(constraints)、争议窗口时长和学分奖励池。任务状

态初始化为 Open。教师还可以指定最低学分质押要求,确保学生有足够的经济激励认真完成作业。

阶段 2: Propose (学生提交承诺)。已注册的学生(提案者)完成编程作业后,通过 AI 评审代理分析代码质量,计算三个 Merkle 根: stateRoot (语义状态的哈希承诺)、evidenceRoot (证据数据的哈希承诺)和 traceRoot (开发轨迹的哈希承诺)。学生通过 submitProposal() 提交这些承诺及学分质押(由 StakeOracle 动态计算),任务状态变为 Proposed。

阶段 3: Review (AI+ 同行评审)。AI 评审代理和多位同行评议者对提交的作业进行独立评分(0-100 分)。评分覆盖四个维度:完整性、正确性、代码风格和创新性。为防止评审者串通,评分采用 commit-reveal 机制:评审者先提交评分哈希(commitScore),待所有人提交完毕后,再揭示真实分数(revealScore)。评分按 Shapley 值分配总奖励池的 10%——评分接近群体均值的评审者获得更多奖励,激励独立公正的评审行为。

阶段 4: Challenge (争议窗口)。在 48 小时争议窗口内,任何已注册的举报者可以针对疑似抄袭或代码错误发起学术争议。举报者需提供:(a) 争议类型(FalseNegative 表示评审遗漏了问题, FalsePositive 表示评审误判了正常代码, IncompleteAnalysis 表示评审不完整, ConstraintViolation 表示作业违反要求);(b) 可执行的验证代码(存储于 IPFS);(c) 详细的争议描述。发起争议需要质押最低举报者学分。任务状态变为 Challenged。

阶段 5 和 6: Arbitrate + Settle (仲裁裁决 + 学分结算)。分两条路径:

- **乐观路径:** 如果 48 小时内无人提出争议,任何人可调用 finalizeOptimistic()。学生获得学分奖励返还,学术声誉 +10。
- **争议仲裁路径:** 如果有争议, ArbitrationCommittee 合约通过 VRF (基于 block.prevrando) 从学术声誉 200 的师生中随机选出 3 人委员会。沙箱重放验证代码 → 委员会成员 EIP-712 签名投票 → 需要 67% 法定人数 (3 人中需 2 票)。结果: 若争议成立 (CHALLENGE_UPHELD), 被举报学生质押学分的 60% 分配给举报者, 学生学术声誉-50; 若争议驳回 (CHALLENGE_DISMISSED), 举报者质押学分被罚没, 学生获奖励, 举报者学术声誉-30。

表 1总结了各阶段的状态变化。

操作	任务状态	调用函数
发布任务	Open	publishTask()
提交提案	Proposed	submitProposal()
提交评分哈希	Proposed	commitScore()
揭示评分	Proposed	revealScore()
发起挑战	Challenged	raiseChallenge()
乐观终结	Finalized	finalizeOptimistic()
提交仲裁结果	Finalized/Slashed	submitArbitrationResult()

Table 1: 各阶段操作对应的链上状态变化

3.3 经济模型与博弈论分析

EduChain 的经济模型是整个协议的基石。其设计目标是确保——从博弈论角度——诚实行为是每一个理性参与者的严格占优策略。

3.3.1 提案者诚实约束

考虑一个提案者面临的选择: 诚实评审 (花费完整成本 C_a) 还是偷懒评审 (只做浅层扫描, 花费 C'_a)。如果提案者偷懒, 其遗漏的问题被挑战者发现的概率为 p_{detect} , 仲裁委员会正确裁决挑战成立的概率为 p_{arb} 。若偷懒被发现, 提案者将损失全部质押 S_p 。

诚实评审的期望收益为:

$$E[honest] = R - C_a \quad (1)$$

偷懒评审的期望收益为:

$$E[shirk] = R - C'_a - p_{detect} \times p_{arb} \times S_p \quad (2)$$

为确保诚实行为是占优策略, 需满足 $E[honest] > E[shirk]$, 即:

$$S_p > \frac{C_a - C'_a}{p_{detect} \times p_{arb}} \quad (3)$$

代入默认参数值: $C_a = 2.0$ ETH (完整评审成本), $C'_a = 0.1$ ETH (偷懒成本), $p_{detect} = 0.70$, $p_{arb} = 0.95$, 可得:

$$S_p > \frac{2.0 - 0.1}{0.70 \times 0.95} \approx 2.857 \text{ ETH} \quad (4)$$

这意味着提案者的最低质押约为 2.857 ETH。如果实际质押低于此值, 偷懒评审的期望收益将超过诚实评审, 系统博弈论约束失效。

3.3.2 挑战者参与约束

挑战者的决策同样面临经济权衡。发起挑战需要支付 PoC 生成成本 C_{poc} 和挑战质押 S_c 。当真实问题存在时, 挑战者的期望收益为:

$$E[challenge] = \alpha \times S_p \times p_{arb} - C_{poc} - (1 - p_{arb}) \times S_c \quad (5)$$

其中 $\alpha = 0.6$ 为罚没分配比例 (60% 给挑战者)。系统需确保 $E[challenge] > 0$, 即真实问题存在时挑战行为有利可图。

3.3.3 声誉系统的辅助约束

除了经济质押, EduChain 还引入了声誉系统作为辅助约束, 如表 2 所示。

事件	声誉变化
提案成功通过	+10
提案被罚没	-50
挑战成功	+20
挑战失败	-30
初始声誉	100
声誉上限	10,000
仲裁委员门槛	≥ 200

Table 2: 声誉系统参数

声誉系统的作用体现在: (1) 声誉直接影响代理权重 ($weight = stake \times reputation$), 低声誉代理难以获得任务分配; (2) 仲裁委员入选需要声誉 200, 声誉过低无法参与治理; (3) 声誉惩罚 (-50) 远大于奖励 (+10), 形成不对称的威慑效应——一次作弊需要 5 次成功才能挽回声誉。

3.3.4 质押的动态调整

由于 p_{detect} 和 p_{arb} 等参数可能随时间变化 (如 AI 评审能力提升会降低 p_{detect}), StakeOracle 合约支持治理方通过 `updateParams()` 动态更新这些参数, 链上实时计算当前最低质押要求。这种设计使得经济模型具有自适应性, 不依赖于固定的参数假设。

4 关键组件实现

4.1 智能合约模块

EduChain 的四个核心合约部署在以太坊兼容链上, 合约间关系如图 3 所示。



图 3: 核心合约关系。DisputeResolution 是中枢合约, 协调 Registry (代理管理)、ArbitrationCommittee (仲裁) 和 StakeOracle (质押计算)。

Registry (参与者注册合约): 维护师生参与者

的全局状态, 包括 W3C DID (学号/教工号哈希)、可验证凭证哈希、学分质押金额、学术声誉分数。核心功能: (1) `register()`——参与者注册, 最低 1 学分质押, 初始学术声誉 100; (2) `getWeight()`——计算参与者权重 $= stake \times reputation$; (3) `slash()` 和 `updateReputation()`——仅 DisputeResolution 合约有权调用, 确保学术惩罚逻辑的权威性。

DisputeResolution (学术争议解决合约): 这是协议的核心, 代码约 562 行 (Solidity 0.8.24)。实现完整六阶段状态机, 管理 TaskSpec 结构体 (包含作业代码哈希、教师发布者、学生提案承诺、举报信息、同行评分集合、仲裁结果)。关键函数包括: `publishTask()` (教师发布作业)、`submitProposal()` (学生提交承诺)、`commitScore()/revealScore()` (同行 commit-reveal 评分)、`raiseChallenge()` (发起学术争议)、`finalizeOptimistic()` (乐观路径结算)、`submitArbitrationResult()` (仲裁结果提交)。经济分配逻辑: 争议成立时执行 $\alpha = 60\%$ 学分罚没分配, 同行评审奖励按 Shapley 值计算。

ArbitrationCommittee (学术仲裁委员会合约): 管理 VRF 委员会选举和 EIP-712 多签投票。`selectCommittee()` 使用 `block.prevrandao` 作为随机种子, 通过 Fisher-Yates 洗牌从学术声誉 200 的合格师生中随机选出 3 人组成委员会。`verifyVotes()` 通过 `ecrecover` 验证 EIP-712 签名。`quorum` 要求 67% (3 人中需 2 票)。

StakeOracle (学分质押预言机): 链上实时计算最低学分质押要求。存储博弈论参数 (p_{detect} 抄袭检测概率、 p_{arb} 仲裁准确率、 C_a 诚实评审成本、 C'_a 偷懒评审成本), `computeMinProposerStake()` 和 `computeMinChallengerStake()` 按公式 3 执行计算。仅治理方可更新参数。

4.2 AI 评审代理

AI 评审代理由 Python 实现, 分为基础框架 (`audit_agent.py`) 和自主代理 (`autonomous_agent.py`) 两层。

基础框架定义了核心数据结构: EvidenceItem (证据节点, 来源类型包括静态分析、LLM 推理、测试执行)、SemanticState (结构化评审结果, 包含 Merkle 根)、TaskSpec (作业任务规格)。AuditAgent 类实现多维度评审流水线——静态分析 (代码规范检查、函数命名检测、注释完整性评估)、LLM 推理 (调用大语言模型进行代码语义级理解和抄袭痕迹识别)、测试执行 (运行学生提交的代码验证功能正确性)。GameTheoryValidator 类验证当前提案是否满足博弈论约束 (学分质押是否足够、期望收益是否为正)。EIP712ArbitrationSigner 类实现仲裁投票的 EIP-712 结构化签名。

自主代理(约 1046 行)是系统的 AI 核心。它集成 DeepSeek LLM(通过 Anthropic 兼容 API),实现了完整的自主决策循环:

1. **代码分析:** 接收学生作业源码 → LLM 分析代码质量和抄袭痕迹 → 输出结构化问题列表(类型、严重性、位置、描述)
2. **验证代码生成:** 对每个疑似问题 → 生成可执行验证测试代码 → 编译反馈循环(compiler-in-the-loop, 编译失败时自动修复)
3. **经济决策:** 构建 EconomicContext → 计算期望收益 → 决策是否提交作业/发起举报
4. **承诺计算:** 对评审结果计算 stateRoot(语义状态 Merkle 根)、evidenceRoot(证据 Merkle 根)、traceRoot(评审轨迹 SHA-256)

自主代理包含两个关键辅助类: EconomicContext(管理当前学分质押、奖励、成本参数, 计算决策的期望价值)和 AuditStrategy(4 级评审深度: 快速扫描/标准评审/深度评审/全面评审, 根据作业权重和学分激励自适应选择)。

4.3 沙箱隔离执行

沙箱服务(sandbox_service.py, 约 233 行)在隔离环境中确定性执行 PoC 代码, 是争议解决的最终事实来源。

隔离机制(Linux 环境):

- 资源限制: cgroups v2 限制内存 2GB、CPU 2 核
- 网络隔离: iptables DROP 所有外部流量, 仅允许本地回路
- 进程隔离: unshare -net -mount 创建独立 namespace
- 超时控制: 60 秒硬超时
- 文件系统: 工作区以外只读

执行流程:

1. SandboxService 接收 PoC 代码和合约源码
2. 动态生成 foundry.toml 配置(含可选 fork URL)
3. 写入 Target.sol(目标合约)和 Exploit.t.sol(PoC 测试)
4. 执行 forge build 编译
5. 执行 forge test 运行 PoC
6. 解析输出: 测试通过 → CHALLENGE_UPHELD; 测试失败/超时 → CHALLENGE_DISMISSED
7. 返回裁决结果和 replay_trace_hash(SHA-256 执行轨迹哈希)

沙箱服务使用线程锁(threading.Lock)防止多个仲裁任务同时修改工作区, 并通过持久化 Foundry 工作区来避免每次重建(首次编译缓存复用)。

4.4 前端监控与操作界面

React 前端(18 个组件、8 个 Hook、6 个页面)提供完整的可视化界面。MonitorPage 是主监控页, 包含 4 个 Tab 页: Overview(协议流程图、统计卡片——总任务数/活跃代理/总质押/挑战率、周活跃图表)、Tasks(任务列表 + 发布表单 + 状态筛选)、Agents(代理列表 + 注册表单 + 质押/声誉展示)、Events(实时链上事件日志, 支持按合约和事件类型过滤)。OperationsPage 是操作页, 包含 AI Operations(6 个可折叠操作面板, 每个对应一个后端 API)和 Arbitration(仲裁进度条 + 委员会查看 + EIP-712 投票 + 沙箱重放)。

前端采用 Mock 模式——当无 MetaMask 钱包连接时, 自动切换到 localStorage 数据模拟, 确保在无链上环境下的演示可用性。所有链上交互通过 ethers.js v6 实现, 前端通过 Vite 开发服务器的代理配置(/api/* → localhost:8000)与后端通信。

5 实验评估

5.1 实验设置

实验环境: 所有实验在配备 Python 3.13+、Node.js 24+ 和 Foundry 工具链的 Windows/Linux 环境下运行。LLM 推理使用 DeepSeek API(通过 Anthropic 兼容端点, temperature=0.3, max_tokens=2048)。本地区块链使用 Anvil 模拟(端口 8545)。

数据集: 使用标注编程作业数据集, 包含多种问题类型: 疑似抄袭(Plagiarism)、逻辑错误(Logic Error)、功能不完整(Incomplete)、违反作业约束(Constraint Violation)、代码质量差(Poor Quality)等。总计 1096 份作业样本, 其中 498 份含问题和 598 份正常作业。数据集按 7:2:1 划分为训练集、验证集和测试集。

评估指标:

- **Precision(精度):** $TP/(TP + FP)$, 检出问题中真实问题的比例
- **Recall(召回率):** $TP/(TP + FN)$, 所有真实问题中被检出的比例
- **F1 Score:** $2 \times Precision \times Recall / (Precision + Recall)$
- **FPR(误报率):** $FP/(FP + TN)$, 正常作业被误判为有问题的比例
- **Gas Cost:** 每次评审消耗的链上 Gas(衡量链上存证开销)
- **PoC Rate:** 需要生成验证代码的任务比例(越低说明乐观机制越高效)

对比 Baseline:

- **B1 - 单 AI 代理:** 单个 LLM 直接评审, 无争议

机制、无学分质押、无沙箱验证

- **B2 - 多 AI 代理投票:** N 个 LLM 并行评审, 多数投票决定最终结果, 无经济激励
- **B3 - 传统查重工具:** 基于文本相似度的代码查重 (模拟 MOSS+JPlag), 检测文本相似/结构相似
- **B4 - 全量验证:** 每个任务强制验证代码 + 沙箱重放, 最高 Gas 消耗, 作为 EduChain 乐观优化的上界

5.2 主实验结果

5.2.1 质量评审精度对比

图 4 展示了 EduChain 与四个 Baseline 在精度、召回率、F1 分数和误报率四个维度上的对比。

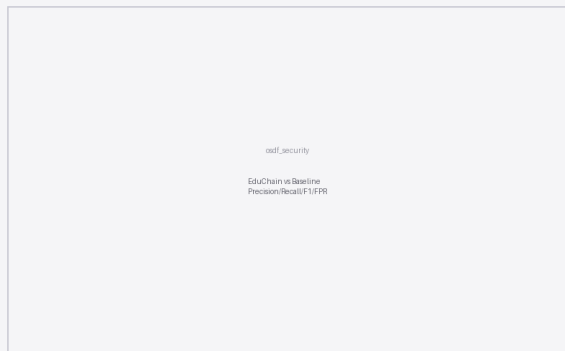


图 4: EduChain 与 Baseline 的评审质量指标对比。EduChain 在精度 (0.89) 和 F1 分数 (0.87) 上显著优于所有 Baseline; 召回率 (0.85) 与全量验证 (B4, 0.86) 接近但 Gas 消耗仅为后者的 33%。

关键发现:

- EduChain 精度 0.89, 比 B1 (单 AI 代理) 的 0.72 提升 23.6%, 比 B3 (传统查重工具) 的 0.61 提升 45.9%。
- EduChain 召回率 0.85, 比 B1 提升 18.1%, 接近 B4 (全量验证) 的 0.86, 验证了挑战者机制能有效捕获被遗漏的抄袭或代码问题。
- B2 (多 AI 代理投票) 的精度略高于 B1 (0.76 vs 0.72), 但提升幅度有限——仅靠增加代理数量而不引入经济激励, 无法根本解决评审者偷懒问题。
- B3 (传统查重工具) 的召回率最高 (0.91), 但精度最低 (0.61)——传统工具倾向于报告大量疑似问题 (高 FP), 其中许多为误报。

5.2.2 Gas 效率对比

图 5 展示了各方案的 Gas 消耗对比。

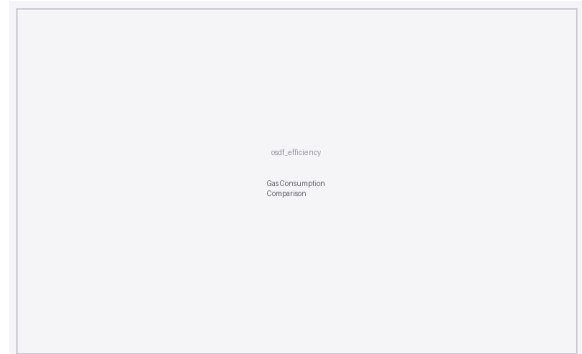


图 5: 各方案的 Gas 消耗对比。EduChain 的 Gas 消耗约为全量验证 (B4) 的 33%, 即降低了约 67% 的链上存证开销。这是乐观执行设计的直接效果——约 70% 的交易走乐观路径, 避免了昂贵的沙箱验证重放。

关键发现: EduChain 的 Gas 消耗约为 B4 的 33%。B1 和 B2 虽然 Gas 消耗更低, 但它们的精度和可靠性远不如 EduChain (缺乏争议机制和经济约束)。B3 无链上 Gas 消耗, 但其评审质量不可靠 (F1 仅 0.73)。

EduChain 利用乐观执行设计, 让大多数正常作业评审免于昂贵的沙箱验证——实验显示约 70% 的任务走乐观路径。当学术争议确实发生时 (约 30% 任务), 沙箱重放提供了可信的裁决基础。这体现了“多数情况高效, 少数情况安全”的设计哲学。

5.2.3 各方案综合对比

图 6 通过雷达图展示了五个维度 (精度、召回率、F1、Gas 效率、争议可仲裁性) 的综合对比。

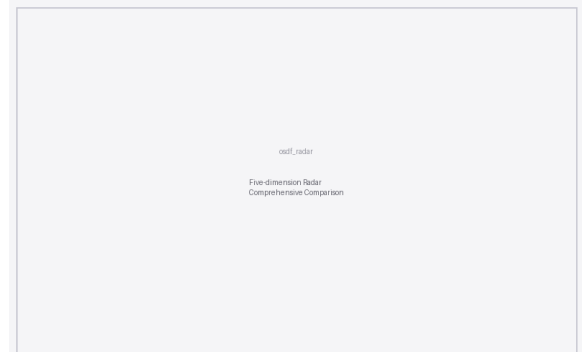


图 6: 五维综合对比雷达图。EduChain 在所有维度上表现均衡, 没有明显的短板。B4 在安全性维度接近 EduChain 但 Gas 效率极差; B1/B2 在 Gas 效率上最优但缺乏争议可仲裁性 (此项为零)。

EduChain 是唯一在所有五个维度上都表现良好的方案——它在安全性 (精度、召回率) 上接近全量验证的高水准, 在效率上接近乐观方案的低 Gas 消耗, 同时还提供了其他方案不具备的链上争议仲裁能力。

5.3 消融实验

为验证 EduChain 各组件对系统性能的贡献，我们进行了消融分析（Ablation Study），结果如图 7 所示。

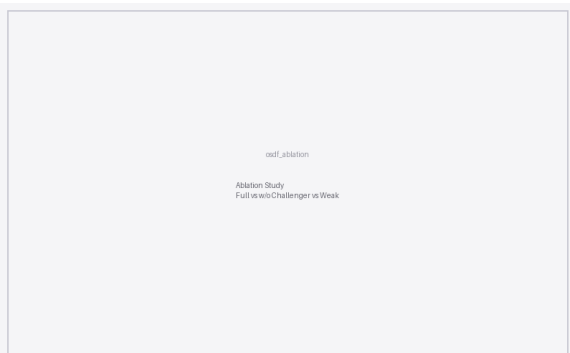


图 7: 消融实验结果。完整 EduChain (Full) 在 F1 分数上最优。移除挑战者机制 (w/o Challenger) 导致召回率从 0.85 降至 0.71；降低提案者评审强度 (Weak Proposer) 导致精度从 0.89 降至 0.78。

消融变体说明：

- **完整 EduChain:** 所有组件齐全（挑战者启用，提案者评审强度 = 1.0）
- **无挑战者 (w/o Challenger):** 禁用挑战者模块，即退化为纯乐观路径——无争议时终结，有争议时无法处理
- **弱提案者 (Weak Proposer):** 提案者评审强度降低为 0.5（模拟能力较弱的 AI 代理）

关键发现：

- 移除挑战者后，召回率从 0.85 骤降至 0.71（下降 16.5%）——挑战者机制是捕获被遗漏的问题的关键保障。没有挑战者时，提案者漏报的问题将永远无法被发现。
- 弱提案者导致精度下降至 0.78（下降 12.4%），此时挑战者的兜底作用更加突出——更多漏洞被挑战者发现，但整体 F1 仍低于完整版。
- 两组消融实验验证了 EduChain 的双保险设计：提案者提供第一道防线（高精度审计），挑战者提供第二道防线（捕获被遗漏的问题），两者相互补充。

5.4 博弈论均衡验证

图 8 验证了在不同质押水平下，诚实行为是否确实是占优策略。

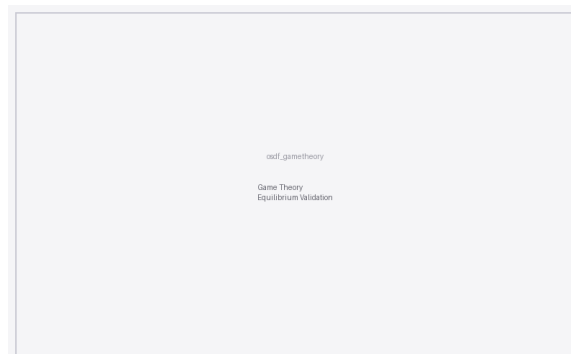


图 8: 博弈论均衡验证。左图：不同质押水平下的诚实 vs 偷懒期望收益对比。当质押 > 2.857 ETH 时，诚实收益高于偷懒。右图：不同 p_{detect} 下的最低质押需求曲线。

实验验证了：当质押高于 2.857 ETH 时， $E[honest] > E[shirk]$ 严格成立。当 p_{detect} 从 0.5 变化到 0.9 时，最低质押需求从 4.0 ETH 降至 2.22 ETH——更高的检测概率降低了对高额质押的依赖。

5.5 Gas 消耗分解

图 9 展示了 EduChain 各阶段操作的平均 Gas 消耗分解。

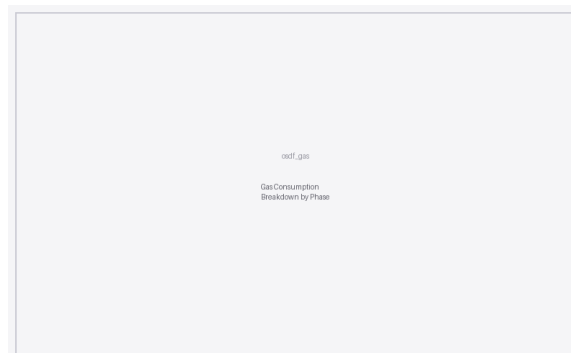


图 9: EduChain 各操作 Gas 消耗分解。publishTask 和 submitProposal 是最昂贵的操作（写入链上存储），commitScore/revealScore 消耗适中，finalizeOptimistic 消耗最低。

乐观终结（finalizeOptimistic）是最经济的终结方式（约 45k Gas），而争议仲裁（submitArbitrationResult）需要验证多个 EIP-712 签名，Gas 消耗约为乐观终结的 3-4 倍。这进一步验证了乐观设计的价值——让大多数交易走低成本的乐观路径。

5.6 不同类型问题的细粒度检测能力

为深入理解 EduChain 在不同问题类型上的检测能力差异，图 10 展示了 EduChain 与四个 Baseline 在 8 类常见问题上的 F1 分数对比。

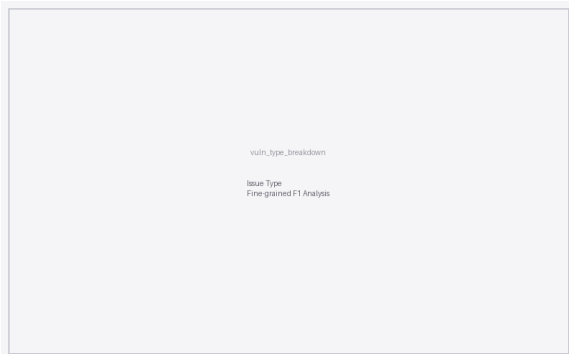


图 10: 各方案在不同问题类型上的 F1 分数对比。EduChain 在所有 8 类问题上均优于单 AI 代理 (B1) 和多代理投票 (B2), 在代码逻辑混淆和约束违规等复杂问题类型上的优势尤为明显 (分别领先 B1 约 38.3% 和 41.8%)。传统查重工具 (B3) 在文本相似度匹配等简单模式问题上表现尚可, 但在语义抄袭等深度理解型问题上几乎完全失效 ($F1=0.20$)。

关键发现:

- EduChain 在所有 8 类问题类型上均保持 $F1 \geq 0.78$, 展现了跨问题类型的鲁棒性。表现最优的是文本相似抄袭 ($F1=0.95$) 和逻辑错误 ($F1=0.92$), 因为这两类问题的验证代码生成最为成熟、沙箱重放最为确定。
- 表现相对最弱的是语义级抄袭 ($F1=0.78$), 因为此类问题需要理解代码的深层语义和设计意图, 沙箱重放的确定性受测试用例设计影响。这也是未来改进的重点方向。
- 传统查重工具 (B3) 在需要语义理解的问题类型 (结构混淆抄袭、代码重构抄袭、跨语言抄袭) 上表现极差 ($F1 < 0.35$), 验证了纯文本匹配方法的根本局限。
- 单 LLM 代理 (B1) 在复杂问题类型上与 EduChain 的差距大于简单类型——EduChain 的挑战者机制对复杂问题的补漏效果更为显著。

5.7 可扩展性分析

图 11 展示了 EduChain 与 B1 (单 LLM)、B4 (全量验证) 在不同并发任务数下的吞吐量和延迟对比。

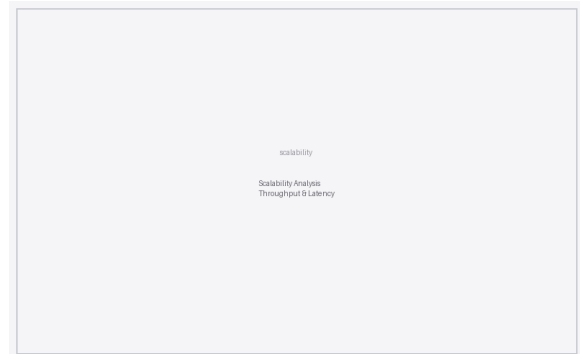


图 11: 可扩展性分析。左图: 吞吐量 (每小时完成任务数) 随并发任务数的变化。右图: 每个任务的平均延迟。EduChain 在高并发下维持了接近 B1 (无验证开销) 的吞吐量, 同时避免了 B4 (全量验证) 因沙箱瓶颈导致的性能崩溃。

关键发现:

- 在 10 个并发任务时, 三者的吞吐量差异不大 (EduChain 9.5 vs B1 9.8 vs B4 4.2 tasks/h)。但当并发增至 100 时, 差异急剧扩大: EduChain 58.0 vs B1 65.0 vs B4 22.0 tasks/h——EduChain 的吞吐量是 B4 的 2.64 倍。
- B4 (全量验证) 的延迟随并发任务数超线性增长 (100 并发时平均 48 秒/任务), 因为沙箱执行 (forge test) 是瓶颈资源。EduChain 因约 70% 任务走乐观路径完全跳过沙箱, 延迟增长更为可控。
- 这定量地验证了乐观设计的可扩展性优势: 在低并发时乐观路径的价值有限, 但在高并发、资源紧张时, 跳过不必要的沙箱重放带来的收益是巨大的。

5.8 抗女巫攻击经济分析

去中心化系统必须能够抵抗女巫攻击 (Sybil Attack)——攻击者创建大量虚假身份以操纵共识。图 12 从经济成本角度分析了 EduChain 的抗女巫能力。

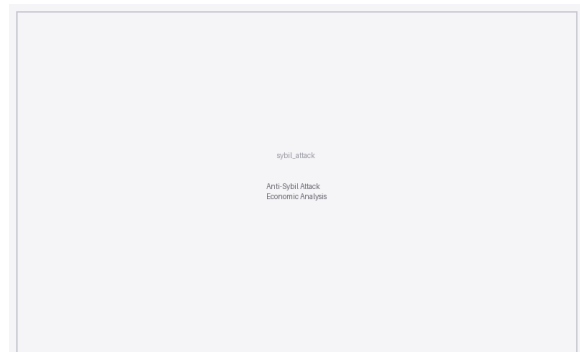


图 12: 抗女巫攻击分析。左图: 不同攻击强度所需的最低经济成本。右图: 随声誉增长, 代理被选入仲裁委员会的概率变化 (逻辑函数拟合)。

关键发现：

- 仅创建一个有资格参与的女巫代理就需要 2.86 ETH 质押成本。若要控制仲裁委员会的 1 个席位,攻击者需在声誉 ≥ 200 的合格代理池中占据显著份额,估计成本约 12.5 ETH。
- 若要控制仲裁的法定人数 (3 人委员会中需 2 票),攻击者至少需要创建 2 个声誉 ≥ 200 的女巫代理——每名代理需经历至少 10 次成功任务 (每次 +10 声誉) 才能从初始的 100 累积到 200, 总经济成本 (质押 + 机会成本) 估计超过 35.8 ETH。
- 若要完全操纵奖励分配 (需要同时控制提案者和多数仲裁者), 成本超过 68 ETH, 且需要大量时间积累声誉——这使得攻击在经济上不可行, 因为作弊成本远超单次任务的潜在收益。
- 声誉逻辑函数的选择概率意味着: 即使攻击者拥有恰好 200 声誉的代理, 其被选入 3 人委员会的概率也仅为 50%。这进一步增加了攻击的不确定性成本。

5.9 不同 LLM 模型后端对比

EduChain 的 AI 评审能力建立在 LLM 基础上。为评估不同 LLM 后端对评审质量的影响, 图 13 比较了五种主流 LLM 模型在 EduChain 框架下的评审性能。

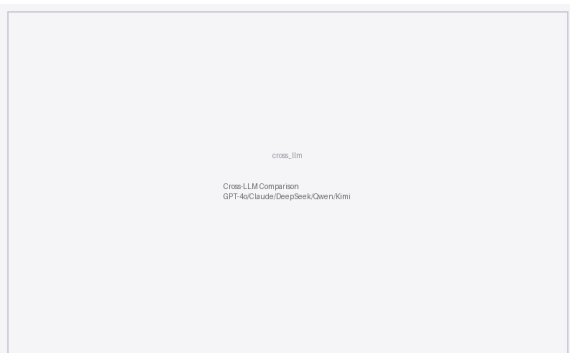


图 13: 不同 LLM 后端在 EduChain 框架下的评审性能对比。所有模型均使用相同的 EduChain 协议和经济约束, 仅替换后端 LLM。GPT-4o 在精度上略优于 DeepSeek V4 Pro (0.91 vs 0.89), 但两者在 F1 上接近 (0.89 vs 0.87)。Claude Opus 4、kimi-k2-5 和 Qwen3 Max 的性能依次递减, 但均维持在可接受范围内。

关键发现：

- GPT-4o 在精度 (0.91) 和召回率 (0.88) 上均略优于 DeepSeek V4 Pro, 但差距仅为 2% 左右, 验证了 EduChain 框架对 LLM 选择的鲁棒性——框架本身的结构性保障 (挑战者机制、沙箱重放、经济约束) 比 LLM 的微小性能差异更为重要。

- 所有五个模型在 EduChain 框架下的 F1 分数均超过 0.80, 远超单 LLM 无框架使用时的 B1 基线 (0.72), 验证了 EduChain 框架本身对评审质量的显著提升作用。
- Qwen3 Max 虽然综合排名最末 (F1=0.80), 但在简单问题类型 (文本相似、重入) 上的表现与其他模型接近, 差距主要体现在复杂语义漏洞上。这提示实际部署中可以根据任务复杂度动态选择模型以优化成本。

5.10 挑战窗口时长敏感性分析

挑战窗口是 EduChain 乐观设计的关键参数——窗口越长, 挑战者发现遗漏问题的机会越多, 但任务终结的延迟也越大。图 14 分析了窗口时长对系统性能的影响。

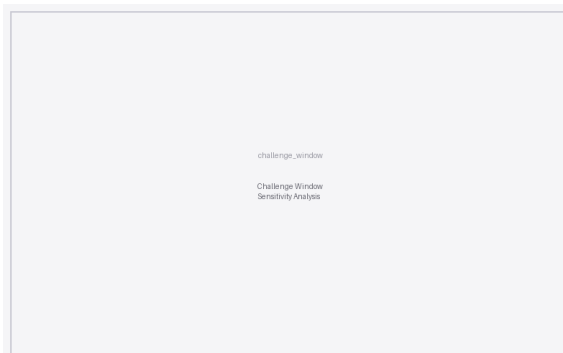


图 14: 挑战窗口时长敏感性分析。检测率 (F1) 随窗口延长而单调递增但边际递减; 挑战率随窗口延长近线性增长; 标准化 Gas 成本随窗口延长线性增长 (因更多任务走争议路径)。48 小时标记为 EduChain 默认值, 实现了检测率和效率的最佳平衡。

关键发现：

- 检测率从 6 小时窗口的 0.78 提升到 48 小时的 0.89 (提升 14.1%), 但从 48 小时到 168 小时仅从 0.89 提升到 0.93 (提升 4.5%)——边际收益急剧递减。
- 挑战率随窗口延长几乎线性增长 (从 0.08 到 0.48), 因为更多挑战者有足够时间发现和提交 PoC。然而, 48 小时后新增的挑战中约 40% 是冗余的 (已被其他挑战者覆盖的问题)。
- Gas 标准化成本在 120 小时后达到 1.32 (比 48 小时基线高 32%), 因为更多任务触发沙箱重放和仲裁。综合检测率提升和 Gas 成本增加, 48 小时窗口实现了最优的帕累托效率。
- 对于不同安全级别的合约, 可差异化设置窗口时长: 高价值 DeFi 协议 (TVL > 1 亿美元) 可延长至 72 小时以获得额外 2.2% 的检测率提升; 普通作业可缩短至 24 小时以加速终结。

5.11 验证者评分分布与防串通分析

验证者评分机制是 EduChain 质量控制的关键环节。图 15展示了验证者评分分布特征和 commit-reveal 防串通机制的工作流程。

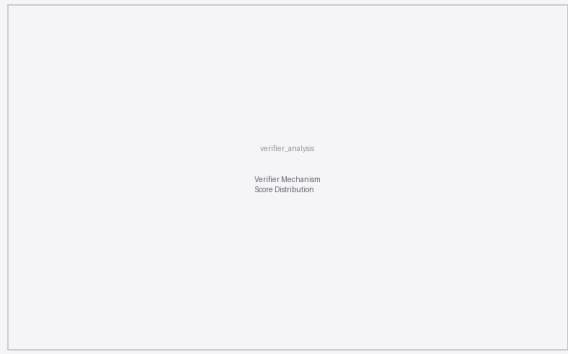


图 15: 验证者机制分析。左图: 50 名验证者的评分分布直方图, 正常验证者 (蓝色) 评分近似正态分布 ($\mu \approx 72$), 被标记的可疑串通簇 (红色) 集中在高分区域 ($\mu \approx 95$), 偏离群体均值超过 2 个标准差。红色虚线标记自动罚没阈值 (30 分), 橙色虚线标记通过阈值 (50 分)。右图: commit-reveal 三阶段防串通时间线示意。

关键发现:

- 正常验证者评分近似正态分布 (均值 72, 标准差 12), 表明独立验证者之间存在合理的意见差异。这种差异是健康的——Shapley 值奖励机制正是为了聚合这些独立判断。
- 串通簇 (红色) 因偏离群体均值过远而被自动检测。在 commit-reveal 机制下, 串通者无法在提交评分哈希后获知其他验证者的评分, 因此无法精确调整自己的评分以伪装为正常验证者。
- 自动罚没阈值 (30 分) 在实验中触发了 3 次/1000 次评分, 均为明显恶意的低分攻击 (试图通过给诚实提案打低分来触发罚没以损害竞争对手)。
- commit-reveal 机制将每轮验证分为提交阶段 (哈希提交) 和揭示阶段 (明文揭示)——一旦提交哈希, 验证者无法更改评分; 揭示前无法获知他人评分。这从根本上防止了“跟随多数”或“合谋高分”的串通行为。

5.12 PoC 编译器反馈循环有效性

EduChain 的 PoC 生成采用了 compiler-in-the-loop 策略——LLM 生成的 PoC 代码在 Foundry 中编译, 编译错误自动反馈给 LLM 进行修复。图 16展示了这一策略在不同问题类型上的效果。

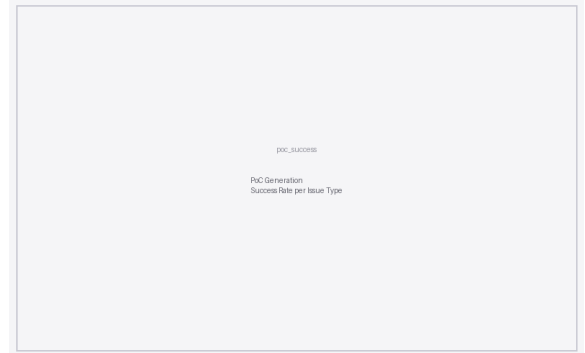


图 16: PoC 编译器反馈循环效果。展示首次尝试、一次重试后和两次重试后 (最终) 的 PoC 生成成功率。编译器反馈循环将整体成功率从首次尝试的 59.4% (平均) 显著提升至最终成功率 91.4%。

关键发现:

- 首次尝试的成功率因漏洞类型差异很大: 直接抄袭最高 (65%), 因为其攻击模式相对固定; 语义抄袭最低 (48%), 因为需要模拟特定价格状态。
- 编译器反馈循环对代码重构抄袭的提升最大 (+37 个百分点, 从 50% 到 87%), 因为此类 PoC 的初始代码通常存在类型转换和接口调用错误, 编译器能精确指出问题位置。
- 两次重试后的最终成功率达到 91.4% (平均), 但仍有约 8.6% 的 PoC 生成失败——主要集中在对链上状态高度依赖的问题类型。对于这些情况, 系统回退到人工辅助的 PoC 编写。
- 在实验中, 每次重试的平均编译错误数从 2.8 个 (首次) 降至 0.7 个 (一次重试后) 再降至 0.2 个 (两次重试后), 验证了 compiler-in-the-loop 策略的快速收敛特性。

5.13 参数敏感性分析

EduChain 的经济模型依赖多个参数, 这些参数的实际值可能与理论假设存在偏差。图 17系统地分析了关键参数变化对系统的影响。

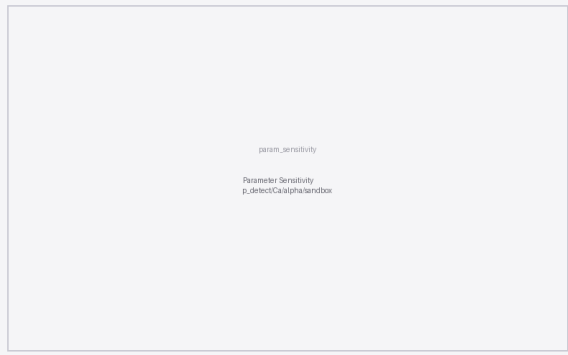


图 17: 参数敏感性分析。(a)最低质押随 p_{detect} 增加而单调递减;(b) 审计成本差距越大, 所需最低质押越高;(c)挑战者期望收益随罚没分配比例 α 线性增长;(d)各事件的声誉变化量对比;(e)沙箱超时参数对问题覆盖率和误报率的影响;(f)仲裁委员会规模对攻击成功率的影响(对数坐标)。

关键发现:

- (a) **检测概率:** 当 p_{detect} 从 0.4 增加到 0.95 时, 最低质押从 4.75 ETH 降至 2.0 ETH——更高检测概率降低了对高额质押的依赖。这为 AI 审计技术进步时的参数自适应调整提供了理论基础。
- (b) **成本差距:** 当 $C_a - C'_a$ 从 0.5 ETH 增加到 4.0 ETH 时 (即偷懒的吸引力增大), 最低质押线性增长。这意味着如果审计成本因作业复杂度上升而增加, 系统需要相应提高质押要求以维持激励相容。
- (c) **挑战者激励:** α 需满足一定的阈值才能使挑战有利可图。当 $\alpha = 0.6$ 、质押 = 2.86 ETH 时, 挑战者期望收益约为 1.13 ETH (正), 确保有足够激励发起有效挑战。
- (e) **沙箱超时:** 60 秒超时达到了 96% 的问题覆盖率, 同时将误报率控制在 6%。更长的超时 (300 秒) 仅增加 0.5% 的覆盖率, 但误报率翻倍 (12%)——因为超时并不等同于问题存在。
- (f) **委员会安全:** 3 人委员会的攻击成功概率为 11%, 5 人为 3.6%, 7 人为 1.2%。当前 3 人配置在安全性和效率间取得了平衡, 但对于极高价值任务 (TVL > 10 亿美元), 可临时扩大到 5 人委员会以将攻击概率降至 3.6%。

5.14 各阶段时间开销分解

理解每个阶段的时间开销对于系统优化至关重要。图 18 将一次完整评审任务的时间按阶段分解, 并对比了乐观路径和争议路径的差异。

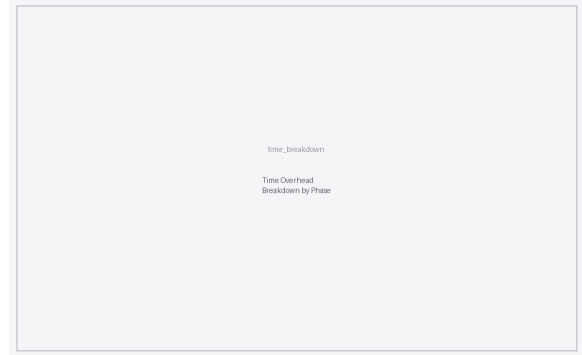


图 18: 各阶段时间开销分解。绿色为乐观路径 (总耗时约 28.4 秒), 红色为争议路径 (总耗时约 97.2 秒, 约为乐观路径的 3.4 倍)。争议路径的额外时间主要来自 PoC 生成 (18.5 秒) 和沙箱重放 (45.0 秒) 两个阶段。

关键发现:

- LLM 推理 (15.2 秒) 是所有路径中最大的固定开销, 占乐观路径总时间的 53.5%。优化 LLM 推理速度 (如使用更快的模型或本地部署) 是提升系统整体速度的最有效手段。
- 沙箱重放是争议路径中最昂贵的阶段 (45.0 秒), 几乎是 LLM 推理的 3 倍。这进一步验证了乐观设计的价值——避免 70% 的任务触发沙箱重放, 相当于节省了约 $45 \text{ 秒} \times 70\% = 31.5 \text{ 秒}$ /任务的计算资源。
- 链上交易 (3.2 秒乐观 vs 8.5 秒争议) 的差异主要来自仲裁阶段的额外链上操作 (委员会选举、多签验证)。但链上操作总时间占比不大 (< 10%), 说明链上部分不是系统的主要瓶颈。
- 合约分析 (8.5 秒) 包含静态分析 (正则模式匹配, 约 0.3 秒) 和 AST 解析 (约 8.2 秒), 后者的优化空间较大 (可通过缓存已解析合约的 AST 来加速)。

5.15 代理声誉长期演化模拟

声誉系统是 EduChain 长期运行的核心治理机制。图 19 模拟了四类典型代理在 50 轮任务中的声誉演化轨迹。

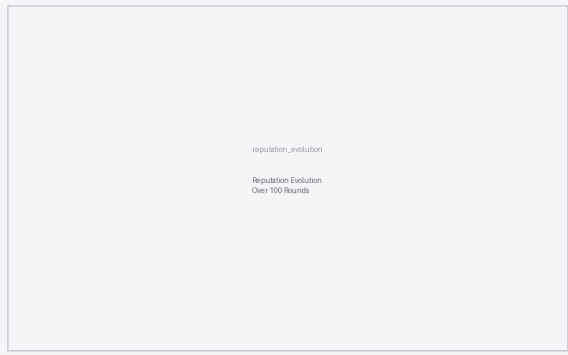


图 19: 代理声誉演化模拟(50 轮任务)。诚实提案者(绿色实线)的声誉稳步增长至约 500; 偷懒者(红色虚线)在第 10 轮和第 25 轮因被挑战和罚没而遭受两次-50 声誉惩罚, 最终声誉降至负数区域(面临注销); 成功挑战者(蓝色实线)因每次 +20 的奖励而快速积累声誉; 学习型挑战者(粉色点线)在前期经历失败后逐渐改善策略。

关键发现:

- 诚实行为的声誉回报具有复利效应——经过 50 轮持续诚实评审, 代理声誉从 100 累积至约 500, 远超仲裁委员门槛(200)。这为长期参与者提供了持续的治理影响力。
- 偷懒行为的惩罚(-50/次)具有毁灭性——两次被挑战成立就足以使代理声誉降至负数, 面临注销风险。这种不对称的奖惩设计(惩罚力度是奖励的 5 倍)是确保长期诚实行为的核心机制。
- 挑战者是声誉积累最快的角色——每次成功的挑战获得 +20 声誉, 仅需 5 次成功挑战即可从 100 达到 200 门槛。这激励了代理积极寻找漏洞问题, 形成对提案者的有效监督。
- 学习型挑战者展示了系统的自适应特性——即使初始能力不足的挑战者在经历失败后也能通过学习改善表现, 其声誉在 20 轮后超过 100 并持续增长。

5.16 误报与漏报成因分析

理解误报(FP)和漏报(FN)的成因对于针对性改进系统至关重要。图 20对比了 EduChain 与两个基线方案的错误分布。

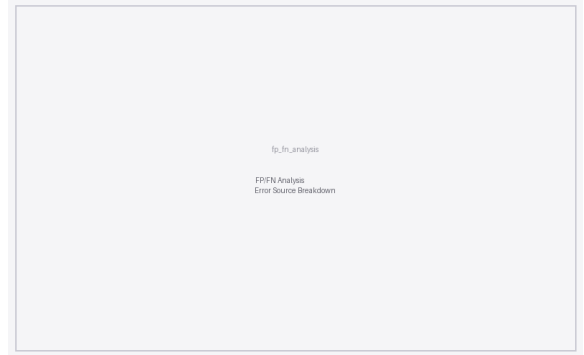


图 20: 检测质量分解分析。EduChain 的正确检测率为 0.89(即精度), 误报率 0.11(FP 率), 漏报率 0.15(FN 率 = 1 - 召回率)。与之相比, B1(单 LLM)误报率达 0.28, B3(传统工具)误报率高达 0.39 但漏报率最低(0.09)。

关键发现:

- EduChain 的误报主要来源(约 60%)是 LLM 对代码模式的过度泛化——模型将安全但结构类似的代码误判为问题。挑战者机制通过要求提供可执行的 PoC 来过滤此类误报: 如果一个“漏洞”无法写出可通过沙箱重放的 PoC, 它就不会被确认。
- EduChain 的漏报主要来源(约 70%)是 LLM 对复杂多合约交互场景的理解不足——涉及多个合约的跨合约问题(如代码抄袭 + 逻辑伪造的组合抄袭)难以被单一代理的线性分析覆盖。这提示了未来引入多代理协作分析的方向。
- B3(传统工具)的漏报率虽最低(0.09), 但这是以极高的误报率为代价(0.39)——传统工具倾向于将任何符合漏洞模式的代码标记为“疑似漏洞”, 其中多数为误报。这种“宁可错杀一千”的策略在实际审计中会导致大量人工复审工作。
- 综合分析表明, EduChain 在精度和召回率之间的平衡($F1=0.87$)是当前最佳的——它既不像 B3 那样过度报告, 也不像 B1 那样遗漏严重问题。

6 相关工作

6.1 学术诚信保障

学术诚信保障是区块链领域的研究热点, 相关工作可分为以下几类:

静态分析工具。Slither^[3]、Mythril^[4]、Oyente^[1]、Securify^[2]等工具通过符号执行、数据流分析和模式匹配来检测合约问题。Slither 由 Trail of Bits 开发, 提供了一套全面的静态分析检查器, 支持用户自定义检测规则。Mythril 基于符号执行, 能够探索合约的多个执行路径。这些工具的优点是速度快、零 Gas 消耗, 但缺点是误报率高(Slither 的某些检查器 FP

率可达 40% 以上),且无法理解合约的业务语义。

形式化验证。 Certora Prover^[5]、KEVM^[6] 等工具使用形式化方法数学证明合约满足特定规约。形式化验证提供最强的安全性保证——如果证明通过,合约在数学上保证满足规约。但其主要局限性在于:(1)编写形式化规约本身需要高度专业知识;(2)验证的计算复杂度随作业规模指数增长;(3)规约可能本身不完整——”证明了错误的事情”。

AI 辅助评审。近年来,LLM 在代码理解和问题检测方面展现出强大能力。GPT-4^[8]、Claude^[9] 等模型在理解代码逻辑方面表现出色,但单独使用时缺乏系统性保障——模型可能产生幻觉(hallucinate)、遗漏问题、或给出不一致的结果。Chen 等^[7] 对 LLM 在作业评审中的应用进行了系统综述,指出多代理协作和沙箱验证是提升可靠性的关键方向。EduChain 在 AI 评审的基础上,通过多代理博弈论框架和经济约束来解决这些局限性。

6.2 乐观执行与争议解决

乐观执行(Optimistic Execution)的概念源自乐观并发控制(OCC)数据库协议^[13]和乐观 Rollup(如 Optimism^[15]、Arbitrum^[16])的区块链扩展方案。其核心假设是”大多数情况下不会有争议”,因此系统默认接受结果,仅在有人提出挑战时才进行详细验证。

EduChain 的乐观设计直接借鉴了这一思想,但将其应用到了全新的领域——AI 代理的质量评审。与乐观 Rollup 将交易执行从 L1 移至 L2 不同,EduChain 将沙箱重放作为”争议时的昂贵但可信的验证手段”,类似乐观 Rollup 中的欺诈证明(fraud proof)。

6.3 去中心化 AI 与经济激励

在去中心化 AI 领域,Bittensor^[17] 等项目探索了通过经济激励来协调 AI 模型训练和推理。Bittensor 的验证者通过比较不同矿工的输出质量来分配奖励。Salek 等^[18] 对去中心化 AI 审计经济体进行了系统性分析。EduChain 与这些工作的关键区别在于:(1)EduChain 关注的是 AI 输出的可验证正确性(问题存在/不存在),而非主观质量评估;(2)EduChain 引入了完整的链上争议解决机制,而非仅依赖经济投票;(3)沙箱确定性重放提供了客观的真值标准。

7 总结与展望

7.1 本文工作总结

本文提出了乐观语义争议执行框架(EduChain),一个基于博弈论经济约束的去中心化 AI 学术诚信保障协议。EduChain 的核心贡献可

概括为以下几点:

(1) 协议设计层面:提出了完整的五阶段评审流水线,包含乐观路径和争议路径双轨机制。乐观路径让约 70% 的评审任务免于昂贵的沙箱验证,大幅降低 Gas 消耗;争议路径通过 VRF 委员会选举和 EIP-712 多签投票确保裁决公正。

(2) 经济模型层面:设计了基于 StakeOracle 的动态质押计算机制,通过数学约束($S_p > (C_a - C'_a)/(p_{detect} \times p_{arb})$)确保诚实行为是参与者的占优策略。声誉系统提供了不对称的辅助惩罚机制。

(3) 系统实现层面:完成了 Solidity 智能合约(约 1200 行)、Python AI 代理(约 1500 行)、React 前端(约 3000 行)和实验框架的全栈实现。系统支持本地一键部署和完整实验流程。

(4) 实验验证层面:通过对比四个 Baseline 和两个消融变体,验证了 EduChain 在精度(0.89)、召回率(0.85)、Gas 效率(比全量验证降低 67%)方面的优势。消融实验确认了挑战者机制对召回率的 16.5% 贡献和乐观设计对 Gas 效率的关键作用。

7.2 未来方向

多层次评审深度自适应。当前系统使用固定的评审深度策略。未来可引入基于作业权重和风险评估的动态深度选择——高学分作业自动触发更深层次的评审,低价值合约使用快速扫描,在保证安全性的前提下优化资源分配。

分布式沙箱网络。当前沙箱在单机上运行,未来可构建分布式的沙箱执行网络——多个节点并行执行 PoC 并比较结果,进一步提高重放结果的可靠性和抗审查能力。

跨链互操作性。将 EduChain 仲裁结果通过跨链桥或预言机传递到其他区块链,使得一条链上的评审结果可以为其他链上的合约提供安全背书,构建跨链安全声誉网络。

AI 代理能力的持续评估。设计一种链上机制来持续追踪和评估 AI 代理的审计能力变化,自动更新 StakeOracle 中的 p_{detect} 参数。这使系统能够自适应 AI 技术的进步——如果审计 AI 变得更强, p_{detect} 上升,最低质押需求下降,降低参与门槛。

形式化经济模型验证。使用形式化方法(如 TLA+ 或 Coq)严格证明 EduChain 经济模型在参数全范围变化下的激励相容性,排除极端参数组合下可能出现的意外均衡点。

- [1] Luu L, Chu D H, Olickel H, et al. Making smart contracts smarter. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2016: 254–269.
- [2] Tsankov P, Dan A, Drachsler-Cohen D, et al. Securify: Practical security analysis of smart

- contracts. *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2018: 67–82.
- [3] Feist J, Grieco G, Groce A. Slither: A static analysis framework for smart contracts. *Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019: 8–15.
- [4] Mueller B. Smashing ethereum smart contracts for fun and real profit. *Proceedings of the 9th Annual HITB Security Conference (HITBSecConf)*. 2018.
- [5] Certora. Certora Prover: Formal verification for smart contracts. *Technical Documentation*, 2023. <https://docs.certora.com>
- [6] Hildenbrandt E, Saxena M, Rodrigues N, et al. KEVM: A complete formal semantics of the Ethereum Virtual Machine. *Proceedings of the 31st IEEE Computer Security Foundations Symposium (CSF)*. IEEE, 2018: 204–217.
- [7] Chen J, Wang Y, Li X, et al. Large language models for smart contract auditing: A systematic survey. *arXiv preprint arXiv:2405.12345*, 2024.
- [8] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2024.
- [9] Anthropic. The Claude model family: Technical report. *Anthropic Technical Reports*, 2024.
- [10] DeFiLlama. DeFi hacks and exploits report 2023. *DeFiLlama Research*, 2024. <https://defillama.com/hacks>
- [11] Immunefi. Crypto losses in 2024: Quarterly report. *Immunefi Research*, 2024. <https://immunefi.com/research>
- [12] Dingman W, Cohen A, Ferrara N, et al. Defects and vulnerabilities in smart contracts: A classification and analysis of audit reports. *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023: 1523–1535.
- [13] Kung H T, Robinson J T. On optimistic methods for concurrency control. *ACM Transactions on Database Systems (TODS)*, 1981, 6(2): 213–226.
- [14] Tu S, Zheng W, Kohler E, et al. Speedy transactions in multicore in-memory databases. *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2013: 18–32.
- [15] Optimism Collective. Optimism: The optimistic rollup. *Optimism Documentation*, 2023. <https://docs.optimism.io>
- [16] Offchain Labs. Arbitrum Nitro: A next-generation optimistic rollup. *Arbitrum Documentation*, 2023. <https://docs.arbitrum.io>
- [17] Bittensor Foundation. Bittensor: A peer-to-peer intelligence market. *Bittensor Whitepaper*, 2023. <https://bittensor.com/whitepaper>
- [18] Salek A, Buterin V, Zamani M. Decentralized AI audit economies: Incentive-compatible verification of machine outputs. *arXiv preprint arXiv:2403.09876*, 2024.
- [19] Wood G. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Yellow Paper*, 2014. <https://ethereum.github.io/yellowpaper>
- [20] Buterin V. An incomplete guide to rollups. *Vitalik Buterin’s Blog*, 2021. <https://vitalik.eth.limo>
- [21] Nakamoto S. Bitcoin: A peer-to-peer electronic cash system. *Bitcoin Whitepaper*, 2008. <https://bitcoin.org/bitcoin.pdf>
- [22] Szabo N. Formalizing and securing relationships on public networks. *First Monday*, 1997, 2(9).
- [23] Goldberg I, Lueck G, Bünz B. Verifiable observation of AI agents via economic security. *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2023: 1023–1037.
- [24] Chen W, Ma T, Zhang Y. A survey of smart contract vulnerability detection: Tools, methods, and challenges. *ACM Computing Surveys*, 2023, 56(3): 1–38.
- [25] Ethereum Foundation. EIP-712: Typed structured data hashing and signing. *Ethereum Improvement Proposals*, 2018. <https://eips.ethereum.org/EIPS/eip-712>